

REST, Interfacce e ACID

Una API HTTP per il KV store distribuito

Architetture dei Sistemi Distribuiti

Lezione 20

Finora il KV store esponeva protocolli testuali su TCP. Ora aggiungiamo un'interfaccia HTTP per discutere:

- risorse;
- rappresentazioni;
- metodi standard;
- codici di stato;
- idempotenza;
- confine tra API pubblica e sistema distribuito sottostante.

REST e' uno stile architetturale:

- risorse identificate da URI;
- rappresentazioni scambiate tra client e server;
- interfaccia uniforme;
- richieste stateless;
- semantica dei metodi nota.

Il punto didattico e':

l'API REST rende pubblico un contratto

Nel lab esponiamo:

- /kv
- /kv/{key}
- /kv/{key}/location
- /cluster/status
- /cluster/shards
- /cluster/rebalance

Non sono semplici nomi. Sono il modo in cui il client vede il sistema.

Comando	REST
GETV key	GET /kv/{key}
SET key value	PUT /kv/{key}
CAS key v value	PATCH /kv/{key}
DELETE key	DELETE /kv/{key}
WHERE key	GET /kv/{key}/location
ADD_SHARD ...	POST /cluster/shards
REBALANCE	POST /cluster/rebalance

GET legge una rappresentazione. Esempio:

```
GET /kv/course
```

Proprietà':

- safe rispetto al valore;
- potenzialmente cacheable;
- non dovrebbe modificare la risorsa.

Nel nostro lab disabilitiamo cache HTTP: topologia e valori possono cambiare.

PUT sostituisce la rappresentazione della risorsa. Esempio:

```
PUT /kv/course
```

```
{"value": "ads"}
```

In REST, PUT dovrebbe essere idempotente rispetto allo stato finale. Domanda: se ripetere PUT incrementa la versione, e' ancora idempotente?

PATCH espone una modifica condizionale.

```
PATCH /kv/course
```

```
{"expected_version": 3, "value": "new"}
```

Questo e' CAS via REST. Se la versione e' vecchia:

```
409 Conflict
```


DELETE elimina una risorsa:

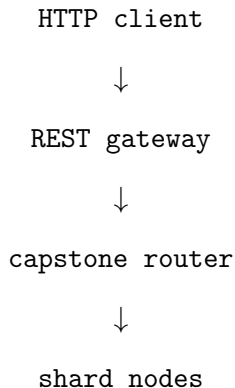
```
DELETE /kv/course
```

POST viene usato per procedure o creazione subordinata:

- POST /cluster/shards
- POST /cluster/rebalance

REBALANCE non e' una semplice sostituzione di risorsa: e' una procedura amministrativa.

Codice	Significato nel lab
200	lettura o update riuscito
201	chiave o shard creato
202	rebalance accettato
204	delete riuscita senza body
400	richiesta malformata
404	chiave assente
409	conflitto di versione
502	router o shard non raggiungibile



Il gateway traduce. Non rende automaticamente ACID il sistema sottostante.

Esempio: Scrittura

```
curl -X PUT http://127.0.0.1:6470/kv/course \  
  -H 'Content-Type: application/json' \  
  -d '{"value": "ads"}'
```

Risposta:

```
{"key": "course", "version": 0, "shard": "S1"}
```

Esempio: CAS Via REST

```
curl -X PATCH http://127.0.0.1:6470/kv/course \  
-H 'Content-Type: application/json' \  
-d '{"expected_version": 0,  
    "value": "distributed-systems"}'
```

Se la versione non coincide:

409 Conflict

ACID non e' una parola magica. Serve a chiedere:

- cosa e' atomico?
- quali invarianti preserviamo?
- cosa vedono operazioni concorrenti?
- cosa sopravvive al crash?

Nel KV store distribuito ogni domanda va localizzata: singola chiave, shard, router, cluster.

Atomicity:

tutto o niente

Nel nostro sistema:

- CAS deve essere atomica sulla singola chiave;
- SET aggiorna valore e versione insieme;
- REBALANCE e' un protocollo composto.

Domanda: IMPORT_KEY riesce e DELETE_LOCAL fallisce. Che stato abbiamo?

Consistency significa preservare invarianti. Esempi:

- versione monotona per chiave;
- CAS vecchia fallisce;
- dopo REBALANCE, routing e posizione reale sono coerenti;
- una risposta con `version` corrisponde al valore restituito.

"Consistenza" senza invariante dichiarato e' ambigua.

Isolation:

cosa vedono operazioni concorrenti?

Nel gateway:

- due PATCH con stessa versione non devono riuscire entrambe;
- non c'è isolamento multi-key;
- durante migrazione bisogna dichiarare stati intermedi osservabili.

CAS aiuta. Non è una transazione generale.

Durability:

una scrittura confermata sopravvive ai guasti previsti

Nel lab REST:

- gli shard sono in memoria;
- nessun WAL;
- nessuno snapshot;
- nessuna recovery dopo crash.

Quindi non dobbiamo promettere durabilita' che non implementiamo.

Per estendere ACID oltre una singola chiave servono meccanismi piu' pesanti:

- lock distribuiti;
- log stabile;
- two-phase commit;
- consenso;
- recovery coordinata.

REST espone il contratto. Non sostituisce questi meccanismi.

- PUT ripetuta e versione che cambia;
- due PATCH concorrenti sulla stessa versione;
- POST /cluster/rebalance lungo o fallito;
- trasferimento multi-key senza transazione;
- promessa falsa di durability.

Ogni scenario separa API, contratto e implementazione.

Avvio del gateway:

```
python3 labs/kv_store/rest_gateway/rest_gateway.py \  
  --port 6470 --router-port 6460
```

Test automatico:

```
python3 labs/kv_store/rest_gateway/acceptance_test.py
```

Una API REST ben disegnata non nasconde i problemi distribuiti. Li rende espliciti:

- cosa e' una risorsa;
- quali operazioni sono safe o idempotenti;
- quali conflitti sono previsti;
- quali proprieta' ACID valgono davvero;
- quali garanzie restano fuori contratto.